BombLab 实验报告

phase_1

运行

```
objdump -d bomb --disassemble=phase_01
```

返回汇编代码如下图。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=phase_01

0000000000402090 <phase_01>:
  402090:       f3 0f 1e fa             endbr64
  402094:       55                      push   %rbp
  402095:       48 89 e5                mov    %rsp,%rbp
  402098:       41 54                   push   %r12
  40209a:       49 89 f4                mov    %rsi,%r12
  40209d:       be 01 00 00 00          mov    $0x1,%esi
  4020a2:       53                      push   %rbx
  4020a3:       48 89 fb                mov    %rdi,%rbx
  4020a6:       48 83 ec 10             sub    $0x10,%rsp
  4020aa:       e8 a1 0b 00 00          call   402c50 <valid_u32_fields>
  4020af:       84 c0                   test   %al,%al
  4020b1:       75 0d                   jne    4020c0 <phase_01+0x30>
  4020b3:       48 83 c4 10             add    $0x10,%rsp
  4020b7:       31 c0                   xor    %eax,%eax
  4020b9:       5b                      pop    %rbx
  4020ba:       41 5c                   pop    %r12
  4020bc:       5d                      pop    %rbp
  4020bd:       c3                      ret
  4020be:       66 90                   xchg   %ax,%ax
  4020c0:       31 c0                   xor    %eax,%eax
  4020c2:       48 8d 55 ec             lea    -0x14(%rbp),%rdx
  4020c6:       be 89 31 40 00          mov    $0x403189,%esi
  4020cb:       48 89 df                mov    %rbx,%rdi
  4020ce:       e8 0d f1 ff ff          call   4011e0 <__isoc99_sscanf@plt>
  4020d3:       83 f8 01                cmp    $0x1,%eax
  4020d6:       75 db                   jne    4020b3 <phase_01+0x23>
  4020d8:       4c 89 e7                mov    %r12,%rdi
  4020db:       e8 70 ff ff ff          call   402050 <p1_compute>
  4020e0:       39 45 ec                cmp    %eax,-0x14(%rbp)
  4020e3:       0f 94 c0                sete   %al
  4020e6:       48 83 c4 10             add    $0x10,%rsp
  4020ea:       5b                      pop    %rbx
  4020eb:       41 5c                   pop    %r12
  4020ed:       5d                      pop    %rbp
  4020ee:       c3                      ret
```

注意到，`4010db`处有函数调用，推测为核心计算函数。运行

```
objdump -d bomb --disassemble=p1_compute
```

返回汇编代码如下图。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=p1_compute

bomb:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000402050 <p1_compute>:
  402050:       f3 0f 1e fa             endbr64
  402054:       55                      push   %rbp
  402055:       48 89 f8                mov    %rdi,%rax
  402058:       48 89 fa                mov    %rdi,%rdx
  40205b:       89 fe                   mov    %edi,%esi
  40205d:       48 c1 e8 20             shr    $0x20,%rax
  402061:       41 89 f8                mov    %edi,%r8d
  402064:       48 c1 ea 10             shr    $0x10,%rdx
  402068:       89 c7                   mov    %eax,%edi
  40206a:       48 89 e5                mov    %rsp,%rbp
  40206d:       e8 ce ff ff ff          call   402040 <p1_step2>
  402072:       48 89 d6                mov    %rdx,%rsi
  402075:       44 89 c7                mov    %r8d,%edi
  402078:       89 c1                   mov    %eax,%ecx
  40207a:       e8 a1 ff ff ff          call   402020 <p1_step0>
  40207f:       89 ce                   mov    %ecx,%esi
  402081:       89 c7                   mov    %eax,%edi
  402083:       e8 a8 ff ff ff          call   402030 <p1_step1>
  402088:       5d                      pop    %rbp
  402089:       c3                      ret

Disassembly of section .fini:
```

分析如下

```
#拆包
mov    %rdi,%rax      # %rax = challenge
mov    %rdi,%rdx      # %rdx = challenge
mov    %edi,%esi      # %esi = challenge 的低32位
shr    $0x20,%rax     # %rax 右移 32 位，此时 %rax 里装的是 challenge 的高32位
mov    %edi,%r8d      # %r8d = challenge 的低32位(备用)
shr    $0x10,%rdx     # %rdx 右移16位
mov    %eax,%edi      # %edi = challenge 的高32位

#第一次计算
call   402040 <p1_step2>  # 调用p1_step2
                          # 参数：%edi (challenge 的高 32 位)
                          # 返回值 %eax

mov    %rdx,%rsi      # 第2个参数 = challenge 右移 16 位
mov    %r8d,%edi      # 第1个参数 = challenge 的低 32 位
mov    %eax,%ecx      # 把刚才 p1_step2 算出来的结果存进 %ecx
call   402020 <p1_step0>  # 调用 p1_step0
                          # 参数：%edi (低32位)， %rsi (右移16位)， %rdx(右移16位)
                          # 返回值在 %eax
```

```
mov    %ecx,%esi       # 第2个参数 = 刚才 p1_step2 的结果
mov    %eax,%edi       # 第1个参数 = 刚才 p1_step0 的结果
call   402030 <p1_step1>  # 调用 p1_step1
                          # 返回值在 %eax，即最终答案。
```

注意到嵌套函数`p1_step2``p1_ste1``p1_step0`，查看其汇编代码。

p1_step0

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --disassemble=p1_step0

bomb:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000402020 <p1_step0>:
  402020:       f3 0f 1e fa             endbr64
  402024:       8d 04 7f                lea    (%rdi,%rdi,2),%eax
  402027:       81 c6 7e 34 00 00       add    $0x347e,%esi
  40202d:       31 f0                   xor    %esi,%eax
  40202f:       c3                      ret

Disassembly of section .fini:
```

分析如下

```
lea    (%rdi,%rdi,2),%eax  # %eax = %rdi + %rdi * 2，即 %rdi * 3
add    $0x347e,%esi        # %esi = %esi + 0x347e
xor    %esi,%eax           # %eax = %eax ^ %esi
```

`%rdi`是`low`，`%esi`是`mid`，C代码为`return (low * 3) ^ (mid + 0x347e);`

p1_step1

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/objdump -d bomb --disassemble=p1_step1

bomb:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000402030 <p1_step1>:
  402030:       f3 0f 1e fa             endbr64
  402034:       8d 04 77                lea    (%rdi,%rsi,2),%eax
  402037:       35 49 c6 00 00          xor    $0xc649,%eax
  40203c:       c3                      ret

Disassembly of section .fini:
```

分析如下

```
lea     (%rdi,%rsi,2),%eax  # %eax = %rdi + %rsi * 2
xor     $0xc649,%eax        # %eax = %eax ^ 0xc649
```

`%rdi` 是 `res0`，`%rsi` 是 `res2`，公式为`return (res0 + res2 * 2) ^ 0xc649;`

p1_step2

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/objdump -d bomb --disassemble=p1_step2

bomb:     file format elf64-x86-64


Disassembly of section .init:

Disassembly of section .plt:

Disassembly of section .plt.sec:

Disassembly of section .text:

0000000000402040 <p1_step2>:
  402040:       f3 0f 1e fa             endbr64
  402044:       8d 86 92 f5 00 00       lea    0xf592(%rsi),%eax
  40204a:       31 f8                   xor    %edi,%eax
  40204c:       c3                      ret

Disassembly of section .fini:
```

分析如下

```
lea     0xf592(%rsi),%eax   # %eax = %rsi + 0xf592
xor     %edi,%eax           # %eax = %eax ^ %edi
```

`p1_step2` 中没有给 `%rsi` 赋值，根据 `p1_compute` 中 `mov %edi,%esi` 得出`%rsi` 中存储 `low`，公式为`return (low + 0xf592) ^ high;`

p1代码如下：

```
int solve_phase_1(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;

uint32_t high = (uint32_t)(challenge >> 32);
uint32_t low  = (uint32_t)challenge;
uint32_t mid  = (uint32_t)(challenge >> 16);

uint32_t res2 = (low + 0xf592) ^ high;

uint32_t res0 = (low * 3) ^ (mid + 0x347e);

uint32_t final_ans = (res0 + res2 * 2) ^ 0xc649;

values[0] = final_ans;
count = 1;

return write_answer(out, cap, values, count);
```

编译代码，正确运行。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ make
gcc -I. -std=c11 -O2 -Wall -Wextra -Wpedantic -Werror defuser_main.c defuser.c tutor/phase0_solution.c challenge.c cli.c -o defuser
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
Phase 2 solver is incomplete or failed.
```

拆弹。

```
第 1 关 · 维护站
终端上留着一份维护记录。
你按了一下回车，屏幕亮起，六个指示灯中只有第一个在闪。
林夏把旁边的椅子拖过来：‘我看记录，你看程序。’
终端能读到接入请求，但还没有放行。旧的口令工具已经找不到了，
留下来的程序仍然在等一个合法的输入。

Phase 1 challenge: 0xbeeb8da1658eec67
Input (unsigned decimal integers):
1943466400
Phase 1 defused.

门锁响了一声，维护站开始接收新的记录。
日志里的请求时间各不相同，城市的日期却始终是同一天。
其中一份校时记录来自街那头的钟楼。林夏记下地址，推开门。

[1 已解除] [2 当前] [3 待解除] [4 待解除] [5 待解除] [6 待解除]
```

phase_02

运行

```
objdump -d bomb --disassemble=phase_02
```

返回汇编代码如下图。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=phase_02
00000000004020f0 <phase_02>:
  4020f0:       f3 0f 1e fa             endbr64
  4020f4:       55                      push   %rbp
  4020f5:       48 89 e5                mov    %rsp,%rbp
  4020f8:       41 55                   push   %r13
  4020fa:       49 89 f5                mov    %rsi,%r13
  4020fd:       be 06 00 00 00          mov    $0x6,%esi
  402102:       41 54                   push   %r12
  402104:       49 89 fc                mov    %rdi,%r12
  402107:       53                      push   %rbx
  402108:       48 83 ec 28             sub    $0x28,%rsp
  40210c:       e8 3f 0b 00 00          call   402c50 <valid_u32_fields>
  402111:       84 c0                   test   %al,%al
  402113:       75 13                   jne    402128 <phase_02+0x38>
  402115:       31 db                   xor    %ebx,%ebx
  402117:       48 8d 65 e8             lea    -0x18(%rbp),%rsp
  40211b:       89 d8                   mov    %ebx,%eax
  40211d:       5b                      pop    %rbx
  40211e:       41 5c                   pop    %r12
  402120:       41 5d                   pop    %r13
  402122:       5d                      pop    %rbp
  402123:       c3                      ret
  402124:       0f 1f 40 00             nopl   0x0(%rax)
  402128:       89 c3                   mov    %eax,%ebx
  40212a:       48 8d 45 d4             lea    -0x2c(%rbp),%rax
  40212e:       48 8d 4d c4             lea    -0x3c(%rbp),%rcx
  402132:       be 7a 31 40 00          mov    $0x40317a,%esi
  402137:       50                      push   %rax
  402138:       48 8d 45 d0             lea    -0x30(%rbp),%rax
  40213c:       48 8d 55 c0             lea    -0x40(%rbp),%rdx
  402140:       4c 89 e7                mov    %r12,%rdi
  402143:       50                      push   %rax
  402144:       4c 8d 4d cc             lea    -0x34(%rbp),%r9
  402148:       4c 8d 45 c8             lea    -0x38(%rbp),%r8
  40214c:       31 c0                   xor    %eax,%eax
  40214e:       e8 8d f0 ff ff          call   4011e0 <__isoc99_sscanf@plt>
  402153:       5a                      pop    %rdx
  402154:       59                      pop    %rcx
  402155:       83 f8 06                cmp    $0x6,%eax
  402158:       75 bb                   jne    402115 <phase_02+0x25>
```

```
40215a:       44 89 e8                 mov    %r13d,%eax
40215d:       8b 75 c0                 mov    -0x40(%rbp),%esi
402160:       83 e0 7f                 and    $0x7f,%eax
402163:       83 c0 17                 add    $0x17,%eax
402166:       39 c6                    cmp    %eax,%esi
402168:       75 ab                    jne    402115 <phase_02+0x25>
40216a:       b9 15 00 00 00           mov    $0x15,%ecx
40216f:       ba 01 00 00 00           mov    $0x1,%edx
402174:       4c 89 e8                 mov    %r13,%rax
402177:       8d 34 b6                 lea    (%rsi,%rsi,4),%esi
40217a:       48 d3 e8                 shr    %cl,%rax
40217d:       83 e0 0f                 and    $0xf,%eax
402180:       01 d0                    add    %edx,%eax
402182:       31 f0                    xor    %esi,%eax
402184:       8b 74 95 c0              mov    -0x40(%rbp,%rdx,4),%esi
402188:       39 c6                    cmp    %eax,%esi
40218a:       75 89                    jne    402115 <phase_02+0x25>
40218c:       48 83 c2 01              add    $0x1,%rdx
402190:       83 c1 05                 add    $0x5,%ecx
402193:       48 83 fa 06              cmp    $0x6,%rdx
402197:       75 db                    jne    402174 <phase_02+0x84>
402199:       e9 79 ff ff ff           jmp    402117 <phase_02+0x27>
```

分析汇编代码如下。

```
#读入数据，验证格式。
mov    $0x6,%esi           # 给 6 个数字的空间
...
call   402c50 <valid_u32_fields>
...
mov    $0x40317a,%esi      # 格式字符串
push   %rax                # 参数压栈（第5、第6个数字的地址）
...
call   4011e0 <__isoc99_sscanf@plt> # 解析出 6 个整数，存入栈中
cmp    $0x6,%eax           # 检查是不是真的读到了 6 个数
jne    ...                 # 不是就失败返回

#计算并验证第一个数是否等于(challenge & 0x7f)
mov    %r13d,%eax          # %eax = challenge（低 32 位）
mov    -0x40(%rbp),%esi    # %esi = 输入的第 1 个数
and    $0x7f,%eax          # %eax = challenge & 0x7f
add    $0x17,%eax          # %eax = (challenge & 0x7f) + 0x17
cmp    %eax,%esi           # 比较输入和公式值
jne    ...                 # 不相等就爆炸

#for循环计算并验证第二至六个数
mov    $0x15,%ecx          # %ecx = 21 (移位次数)
mov    $0x1,%edx           # %edx = 1 (循环变量 i)
# 循环开始
mov    %r13,%rax           # %rax = challenge (64位)
lea    (%rsi,%rsi,4),%esi  # %esi = %esi * 5 (即前一个数乘 5)
shr    %cl,%rax            # %rax = challenge >> %ecx (右移)
and    $0xf,%eax           # %eax = (challenge >> %ecx) & 0xf
add    %edx,%eax           # %eax = %eax + i
xor    %esi,%eax           # %eax = (前一个数 * 5) ^ (((challenge >> %ecx) & 0xf) +
i)
```

```
mov    -0x40(%rbp,%rdx,4),%esi # 取出输入的第 i+1 个数
cmp    %eax,%esi           # 比较
jne    ...                 # 不相等就爆炸

add    $0x1,%rdx           # i++
add    $0x5,%ecx           # 移位次数 += 5 (21 -> 26 -> 31 ...)
cmp    $0x6,%rdx           # 循环直到 i == 6
jne    402174              # 跳回循环起点
```

对应C代码为：

```
int solve_phase_2(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 6;

    values[0] = (challenge & 0x7f) + 0x17;

    uint32_t shift = 21;
    for (size_t i = 1; i < 6; ++i) {
        uint32_t prev_mul_5 = values[i - 1] * 5;
        uint32_t part2 = ((challenge >> shift) & 0xf) + i;
        values[i] = prev_mul_5 ^ part2;
        shift += 5;
    }
    count = 6;
    return write_answer(out, cap, values, count);
}
```

编译成功获取密码。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
117 579 2911 14539 72677 363391
Phase 3 solver is incomplete or failed.
```

phase_3

获取汇编代码如下。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=phase_03
0000000000402290 <phase_03>:
  402290:       f3 0f 1e fa             endbr64
  402294:       55                      push   %rbp
  402295:       48 89 e5                mov    %rsp,%rbp
  402298:       41 54                   push   %r12
  40229a:       49 89 f4                mov    %rsi,%r12
  40229d:       be 02 00 00 00          mov    $0x2,%esi
  4022a2:       53                      push   %rbx
  4022a3:       48 89 fb                mov    %rdi,%rbx
  4022a6:       48 83 ec 10             sub    $0x10,%rsp
  4022aa:       e8 a1 09 00 00          call   402c50 <valid_u32_fields>
  4022af:       84 c0                   test   %al,%al
  4022b1:       75 0d                   jne    4022c0 <phase_03+0x30>
  4022b3:       31 c0                   xor    %eax,%eax
  4022b5:       48 83 c4 10             add    $0x10,%rsp
  4022b9:       5b                      pop    %rbx
  4022ba:       41 5c                   pop    %r12
  4022bc:       5d                      pop    %rbp
  4022bd:       c3                      ret
  4022be:       66 90                   xchg   %ax,%ax
  4022c0:       31 c0                   xor    %eax,%eax
  4022c2:       48 8d 4d ec             lea    -0x14(%rbp),%rcx
  4022c6:       48 8d 55 e8             lea    -0x18(%rbp),%rdx
  4022ca:       be 86 31 40 00          mov    $0x403186,%esi
  4022cf:       48 89 df                mov    %rbx,%rdi
  4022d2:       e8 09 ef ff ff          call   4011e0 <__isoc99_sscanf@plt>
  4022d7:       83 f8 02                cmp    $0x2,%eax
  4022da:       75 d7                   jne    4022b3 <phase_03+0x23>
  4022dc:       44 89 e7                mov    %r12d,%edi
  4022df:       31 c0                   xor    %eax,%eax
  4022e1:       83 e7 07                and    $0x7,%edi
  4022e4:       39 7d e8                cmp    %edi,-0x18(%rbp)
  4022e7:       75 cc                   jne    4022b5 <phase_03+0x25>
  4022e9:       4c 89 e0                mov    %r12,%rax
  4022ec:       4c 89 e6                mov    %r12,%rsi
  4022ef:       48 c1 e8 03             shr    $0x3,%rax
  4022f3:       48 c1 ee 20             shr    $0x20,%rsi
  4022f7:       31 c6                   xor    %eax,%esi
  4022f9:       e8 32 ff ff ff          call   402230 <p3_dispatch>
  4022fe:       39 45 ec                cmp    %eax,-0x14(%rbp)
```

```
402301:       0f 94 c0                  sete   %al
402304:       eb af                     jmp    4022b5 <phase_03+0x25>
```

分析汇编代码如下。

```
#检查输入格式
mov     $0x2,%esi           # 期望输入2个数字
call    402c50 <valid_u32_fields>
...
call    4011e0 <__isoc99_sscanf@plt> # 解析出 2 个整数，分别放在 -0x18(%rbp) 和
-0x14(%rbp)
cmp     $0x2,%eax           # 如果解析数量不是 2，失败

#检查第一个数是否和%edi相等
mov     %r12d,%edi          # %r12 是 challenge，%edi 取它的低 32 位
and     $0x7,%edi           # %edi = challenge & 0x7
```

```
cmp    %edi,-0x18(%rbp)    # 比较输入的第 1 个数 和 %edi
jne    ...                 # 不相等就爆炸

#检查第一个数是否和%esi相等
mov    %r12,%rax           # %rax = challenge
mov    %r12,%rsi           # %rsi = challenge
shr    $0x3,%rax           # %rax = challenge >> 3
shr    $0x20,%rsi          # %rsi = challenge >> 32
xor    %eax,%esi           # %esi = (challenge >> 32) ^ (uint32_t)(challenge >>
3)
call   402230 <p3_dispatch> # 调用 p3_dispatch，参数是 %esi
cmp    %eax,-0x14(%rbp)    # 比较输入的第 2 个数 和 p3_dispatch 的返回值
sete   %al                 # 相等
```

答案为 `p3_dispatch((challenge >> 32) ^ (challenge >> 3))`。

运行

```
objdump -d bomb --disassemble=p3_dispatch
```

得到`p3_dispatch`汇编代码如下。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb -
disassemble=p3_dispatch
  402234:       89 f8                   mov    %edi,%eax
  402236:       89 f7                   mov    %esi,%edi
  402238:       83 f8 07                cmp    $0x7,%eax
  40223b:       77 45                   ja     402282 <p3_dispatch+0x52>
  40223d:       3e ff 24 c5 20 58 40    notrack jmp *0x405820(,%rax,8)
  402244:       00
  402245:       0f 1f 00                nopl   (%rax)
  402248:       e9 53 ff ff ff          jmp    4021a0 <p3_case_0>
  40224d:       0f 1f 00                nopl   (%rax)
  402250:       eb be                   jmp    402210 <p3_case_6>
  402252:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
  402258:       e9 73 ff ff ff          jmp    4021d0 <p3_case_3>
  40225d:       0f 1f 00                nopl   (%rax)
  402260:       e9 4b ff ff ff          jmp    4021b0 <p3_case_1>
  402265:       0f 1f 00                nopl   (%rax)
  402268:       e9 73 ff ff ff          jmp    4021e0 <p3_case_4>
  40226d:       0f 1f 00                nopl   (%rax)
  402270:       eb 8e                   jmp    402200 <p3_case_5>
  402272:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
  402278:       e9 43 ff ff ff          jmp    4021c0 <p3_case_2>
  40227d:       0f 1f 00                nopl   (%rax)
  402280:       eb 9e                   jmp    402220 <p3_case_7>
  402282:       31 c0                   xor    %eax,%eax
  402284:       c3                      ret
```

分析汇编代码。

```
mov    %edi,%eax        # %eax = challenge & 0x7 (取值 0~7)
mov    %esi,%edi        # %edi = 算出来的值
cmp    $0x7,%eax        # 检查 %eax 是否大于 7
ja     402282           # 如果大于 7，直接返回 0
notrack jmp *0x405820(,%rax,8) #根据 %rax 的值，去跳转表里找对应地址跳过去。
```

即：根据 `challenge & 0x7` 的数值，决定跳转至哪一个 `p3_case_X` 函数，在此之前将计算参数放入 `%edi`

执行

```
objdump -d bomb | grep -A 8 '<p3_case_'
```

以抓取8个case，得到汇编代码如下

```
00000000004021a0 <p3_case_0>:
  4021a0:       f3 0f 1e fa             endbr64
  4021a4:       8d 84 bf 8a 88 00 00    lea    0x888a(%rdi,%rdi,4),%eax
  4021ab:       c3                      ret
  4021ac:       0f 1f 40 00             nopl   0x0(%rax)

00000000004021b0 <p3_case_1>:
  4021b0:       f3 0f 1e fa             endbr64
  4021b4:       89 f8                   mov    %edi,%eax
  4021b6:       d1 ef                   shr    $1,%edi
  4021b8:       35 db 1c 00 00          xor    $0x1cdb,%eax
  4021bd:       01 f8                   add    %edi,%eax
  4021bf:       c3                      ret

00000000004021c0 <p3_case_2>:
  4021c0:       f3 0f 1e fa             endbr64
  4021c4:       8d 84 bf dc 24 04 00    lea    0x424dc(%rdi,%rdi,4),%eax
  4021cb:       c3                      ret
  4021cc:       0f 1f 40 00             nopl   0x0(%rax)

00000000004021d0 <p3_case_3>:
  4021d0:       f3 0f 1e fa             endbr64
  4021d4:       8d 04 3f                lea    (%rdi,%rdi,1),%eax
  4021d7:       35 b4 5e 00 00          xor    $0x5eb4,%eax
  4021dc:       c3                      ret
  4021dd:       0f 1f 00                nopl   (%rax)

00000000004021e0 <p3_case_4>:
  4021e0:       f3 0f 1e fa             endbr64
  4021e4:       8d 04 bd 00 00 00 00    lea    0x0(,%rdi,4),%eax
  4021eb:       35 a4 1c 00 00          xor    $0x1ca4,%eax
  4021f0:       c3                      ret
  4021f1:       66 66 2e 0f 1f 84 00    data16 cs nopw 0x0(%rax,%rax,1)
  4021f8:       00 00 00 00
  4021fc:       0f 1f 40 00             nopl   0x0(%rax)
```

```
0000000000402200 <p3_case_5>:
  402200:       f3 0f 1e fa             endbr64
  402204:       89 f8                   mov    %edi,%eax
  402206:       d1 ef                   shr    $1,%edi
  402208:       35 8f 4c 00 00          xor    $0x4c8f,%eax
  40220d:       01 f8                   add    %edi,%eax
  40220f:       c3                      ret

0000000000402210 <p3_case_6>:
  402210:       f3 0f 1e fa             endbr64
  402214:       8d 84 bf 3c c5 03 00    lea    0x3c53c(%rdi,%rdi,4),%eax
  40221b:       c3                      ret
  40221c:       0f 1f 40 00             nopl   0x0(%rax)

0000000000402220 <p3_case_7>:
  402220:       f3 0f 1e fa             endbr64
  402224:       8d 84 7f c9 b0 00 00    lea    0xb0c9(%rdi,%rdi,2),%eax
  40222b:       c3                      ret
  40222c:       0f 1f 40 00             nopl   0x0(%rax)

0000000000402230 <p3_dispatch>:
  402230:       f3 0f 1e fa             endbr64
  402234:       89 f8                   mov    %edi,%eax
--
  402248:       e9 53 ff ff ff          jmp    4021a0 <p3_case_0>
  40224d:       0f 1f 00                nopl   (%rax)
  402250:       eb be                   jmp    402210 <p3_case_6>
  402252:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
  402258:       e9 73 ff ff ff          jmp    4021d0 <p3_case_3>
  40225d:       0f 1f 00                nopl   (%rax)
  402260:       e9 4b ff ff ff          jmp    4021b0 <p3_case_1>
  402265:       0f 1f 00                nopl   (%rax)
  402268:       e9 73 ff ff ff          jmp    4021e0 <p3_case_4>
  40226d:       0f 1f 00                nopl   (%rax)
  402270:       eb 8e                   jmp    402200 <p3_case_5>
  402272:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
  402278:       e9 43 ff ff ff          jmp    4021c0 <p3_case_2>
  40227d:       0f 1f 00                nopl   (%rax)
  402280:       eb 9e                   jmp    402220 <p3_case_7>
```

```
  402282:       31 c0                   xor    %eax,%eax
  402284:       c3                      ret
  402285:       66 66 2e 0f 1f 84 00    data16 cs nopw 0x0(%rax,%rax,1)
  40228c:       00 00 00 00

0000000000402290 <phase_03>:
  402290:       f3 0f 1e fa             endbr64
  402294:       55                      push   %rbp
```

实现路径分别为

- `p3_case_0`:
  `lea 0x888a(%rdi,%rdi,4),%eax` → `%eax = 0x888a + x + x*4` → return (x * 5) + 0x888a;`
- `p3_case_1`:
  `mov %edi,%eax` (`%eax = x`)
  `shr $1,%edi` (`%edi = x >> 1`)
  `xor $0x1cdb,%eax` (`%eax = x ^ 0x1cdb`)

`add %edi,%eax` (`%eax = %eax + %edi`)
→ `return (x ^ 0x1cdb) + (x >> 1);`

- `p3_case_2`:
  `lea 0x424dc(%rdi,%rdi,4),%eax` → `return (x \* 5) + 0x424dc;`
- `p3_case_3`:
  `lea (%rdi,%rdi,1),%eax` (`%eax = x * 2`)
  `xor $0x5eb4,%eax`
  → `return (x \* 2) ^ 0x5eb4;`
- `p3_case_4`:
  `lea 0x0(,%rdi,4),%eax` (`%eax = x * 4`)
  `xor $0x1ca4,%eax`
  → `return (x \* 4) ^ 0x1ca4;`
- `p3_case_5`:
  `mov %edi,%eax` (`%eax = x`)
  `shr $1,%edi` (`%edi = x >> 1`)
  `xor $0x4c8f,%eax` (`%eax = x ^ 0x4c8f`)
  `add %edi,%eax`
  → `return (x ^ 0x4c8f) + (x >> 1);`
- `p3_case_6`:
  `lea 0x3c53c(%rdi,%rdi,4),%eax` -> `return (x \* 5) + 0x3c53c;`
- `p3_case_7`:
  `lea 0xb0c9(%rdi,%rdi,2),%eax` -> `%eax = 0xb0c9 + x + x*2` → `return (x \* 3) + 0xb0c9;`

最终代码为

```
int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;

    values[0] = challenge & 0x7;

    uint32_t x = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
    uint32_t ans = 0;

    switch (values[0]) {
        case 0:
            ans = (x * 5) + 0x888a;
            break;
        case 1:
            ans = (x ^ 0x1cdb) + (x >> 1);
            break;
        case 2:
            ans = (x * 5) + 0x424dc;
            break;
        case 3:
            ans = (x * 2) ^ 0x5eb4;
            break;
        case 4:
            ans = (x * 4) ^ 0x1ca4;
            break;
        case 5:
            ans = (x ^ 0x4c8f) + (x >> 1);
            break;
```

```
        case 6:
            ans = (x * 5) + 0x3c53c;
            break;
        case 7:
            ans = (x * 3) + 0xb0c9;
            break;
    }

    values[1] = ans;

    return write_answer(out, cap, values, count);
}
```

编译成功。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ make
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
117 579 2911 14539 72677 363391
3 3625181654
Phase 4 solver is incomplete or failed.
```

phase_04

运行odjump，得到汇编代码如下

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=phase_04

0000000000402380 <phase_04>:
  402380:       f3 0f 1e fa             endbr64
  402384:       55                      push   %rbp
  402385:       48 89 e5                mov    %rsp,%rbp
  402388:       41 54                   push   %r12
  40238a:       49 89 f4                mov    %rsi,%r12
  40238d:       be 01 00 00 00          mov    $0x1,%esi
  402392:       53                      push   %rbx
  402393:       48 89 fb                mov    %rdi,%rbx
  402396:       48 83 ec 10             sub    $0x10,%rsp
  40239a:       e8 b1 08 00 00          call   402c50 <valid_u32_fields>
  40239f:       84 c0                   test   %al,%al
  4023a1:       75 0d                   jne    4023b0 <phase_04+0x30>
  4023a3:       48 83 c4 10             add    $0x10,%rsp
  4023a7:       31 c0                   xor    %eax,%eax
  4023a9:       5b                      pop    %rbx
  4023aa:       41 5c                   pop    %r12
  4023ac:       5d                      pop    %rbp
  4023ad:       c3                      ret
  4023ae:       66 90                   xchg   %ax,%ax
  4023b0:       31 c0                   xor    %eax,%eax
  4023b2:       48 8d 55 ec             lea    -0x14(%rbp),%rdx
  4023b6:       be 89 31 40 00          mov    $0x403189,%esi
  4023bb:       48 89 df                mov    %rbx,%rdi
  4023be:       e8 1d ee ff ff          call   4011e0 <__isoc99_sscanf@plt>
  4023c3:       83 f8 01                cmp    $0x1,%eax
  4023c6:       75 db                   jne    4023a3 <phase_04+0x23>
  4023c8:       4c 89 e1                mov    %r12,%rcx
  4023cb:       4c 89 e0                mov    %r12,%rax
  4023ce:       44 89 e7                mov    %r12d,%edi
  4023d1:       31 f6                   xor    %esi,%esi
  4023d3:       48 c1 e8 20             shr    $0x20,%rax
  4023d7:       48 c1 e9 07             shr    $0x7,%rcx
  4023db:       83 e7 7f                and    $0x7f,%edi
  4023de:       ba 7f 00 00 00          mov    $0x7f,%edx
  4023e3:       31 c1                   xor    %eax,%ecx
  4023e5:       e8 26 ff ff ff          call   402310 <p4_walk>
  4023ea:       39 45 ec                cmp    %eax,-0x14(%rbp)
  4023ed:       0f 94 c0                sete   %al
```

```
 4023f0:       48 83 c4 10             add    $0x10,%rsp
 4023f4:       5b                      pop    %rbx
 4023f5:       41 5c                   pop    %r12
 4023f7:       5d                      pop    %rbp
 4023f8:       c3                      ret
```

注意到调用了 `p4_walk` ，参数与对应寄存器分别为

1. `%edi`：`challenge & 0x7f`
2. `%esi`：`0`
3. `%edx`：`0x7f`
4. `%ecx`：`(challenge >> 32) ^ (challenge >> 7)`

对`p4_walk`执行odjump，得到汇编代码如下

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -d bomb --
disassemble=p4_walk
  402316:       89 d7                   mov    %edx,%edi
  402318:       29 f7                   sub    %esi,%edi
  40231a:       d1 ef                   shr    $1,%edi
  40231c:       01 f7                   add    %esi,%edi
  40231e:       39 c7                   cmp    %eax,%edi
  402320:       74 4e                   je     402370 <p4_walk+0x60>
  402322:       55                      push   %rbp
  402323:       48 89 e5                mov    %rsp,%rbp
  402326:       53                      push   %rbx
  402327:       8d 1c 0f                lea    (%rdi,%rcx,1),%ebx
  40232a:       48 83 ec 08             sub    $0x8,%rsp
  40232e:       39 f8                   cmp    %edi,%eax
  402330:       72 1e                   jb     402350 <p4_walk+0x40>
  402332:       8d 77 01                lea    0x1(%rdi),%esi
  402335:       89 c7                   mov    %eax,%edi
  402337:       e8 d4 ff ff ff          call   402310 <p4_walk>
  40233c:       8d 04 40                lea    (%rax,%rax,2),%eax
  40233f:       31 d8                   xor    %ebx,%eax
  402341:       48 8b 5d f8             mov    -0x8(%rbp),%rbx
  402345:       c9                      leave
  402346:       83 f0 1b                xor    $0x1b,%eax
  402349:       c3                      ret
  40234a:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)
  402350:       8d 57 ff                lea    -0x1(%rdi),%edx
  402353:       89 c7                   mov    %eax,%edi
  402355:       e8 b6 ff ff ff          call   402310 <p4_walk>
  40235a:       8d 04 40                lea    (%rax,%rax,2),%eax
  40235d:       31 d8                   xor    %ebx,%eax
  40235f:       48 8b 5d f8             mov    -0x8(%rbp),%rbx
  402363:       c9                      leave
  402364:       83 f0 0f                xor    $0xf,%eax
  402367:       c3                      ret
  402368:       0f 1f 84 00 00 00 00    nopl   0x0(%rax,%rax,1)
  40236f:       00
  402370:       8d 47 32                lea    0x32(%rdi),%eax
  402373:       31 c8                   xor    %ecx,%eax
  402375:       c3                      ret

Disassembly of section .fini:
```

分析代码。陈列传入参数：

1. `A` (目标值)= `challenge & 0x7f` (存放在 `%eax` 中)
2. `lo` (下界)= 初始为 `0`
3. `hi` (上界) = 初始为 `0x7f`
4. `D` (异或因子) = `(challenge >> 32) ^ (challenge >> 7)`

```
mov     %edx,%edi           # %edi = hi
sub     %esi,%edi           # %edi = hi - lo
shr     $1,%edi             # %edi = (hi - lo) / 2
add     %esi,%edi           # %edi = lo + (hi - lo) / 2  --> 这就是 mid!
cmp     %eax,%edi           # 比较 mid 和 A
je      402370              # 如果 mid == A，跳去算最终结果
```

即求中间值 `mid = lo + (hi - lo) / 2`

若`mid == A`，则执行如下：

```
0x32(%rdi),%eax  # %eax = mid + 0x32
%ecx,%eax        # %eax = (mid + 0x32) ^ D
ret                      # 直接返回
```

若`mid < A`，则执行如下：

```
call   p4_walk             # 递归调用：p4_walk(A, mid + 1, hi, D)
lea    (%rax,%rax,2),%eax  # 返回值 res * 3
xor    %ebx,%eax           # (res * 3) ^ (mid + D)
xor    $0x1b,%eax          # 最终结果再异或 0x1b
ret
```

若`mid > A`，则执行如下：

```
call   p4_walk             # 递归调用：p4_walk(A, lo, mid - 1, D)
lea    (%rax,%rax,2),%eax  # 返回值 res * 3
xor    %ebx,%eax           # (res * 3) ^ (mid + D)
xor    $0xf,%eax           # 最终结果再异或 0xf
ret
```

对应C代码如下：

```c
int solve_phase_4(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 1;
    uint32_t A = challenge & 0x7f;
    uint32_t D = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 7);

    uint32_t lo = 0;
    uint32_t hi = 0x7f;

    uint32_t mids[32];
    int dirs[32];
    int depth = 0;

    while (1) {
        uint32_t mid = lo + (hi - lo) / 2;

        if (mid == A) {
            break;
        }

        mids[depth] = mid;
        if (A > mid) {
            dirs[depth] = 1;
            lo = mid + 1;
        } else {
            dirs[depth] = 0;
            hi = mid - 1;
        }
        depth++;
    }

    uint32_t res = (A + 0x32) ^ D;
```

```
    for (int i = depth - 1; i >= 0; i--) {
        uint32_t mid = mids[i];
        if (dirs[i] == 1) {
            res = ((res * 3) ^ (mid + D)) ^ 0x1b;
        } else {
            res = ((res * 3) ^ (mid + D)) ^ 0xf;
        }
    }

    values[0] = res;
    return write_answer(out, cap, values, count);
}
```

编译执行成功。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
117 579 2911 14539 72677 363391
3 3625181654
2544721993
Phase 5 solver is incomplete or failed.
```

phase_05

运行odjump，得到汇编代码如下

```
0000000000402470 <phase_05>:
  402470:       f3 0f 1e fa             endbr64
  402474:       55                      push   %rbp
  402475:       48 89 e5                mov    %rsp,%rbp
  402478:       41 54                   push   %r12
  40247a:       49 89 f4                mov    %rsi,%r12
  40247d:       be 02 00 00 00          mov    $0x2,%esi
  402482:       53                      push   %rbx
  402483:       48 89 fb                mov    %rdi,%rbx
  402486:       48 83 ec 10             sub    $0x10,%rsp
  40248a:       e8 c1 07 00 00          call   402c50 <valid_u32_fields>
  40248f:       84 c0                   test   %al,%al
  402491:       75 0d                   jne    4024a0 <phase_05+0x30>
  402493:       31 c0                   xor    %eax,%eax
  402495:       48 83 c4 10             add    $0x10,%rsp
  402499:       5b                      pop    %rbx
  40249a:       41 5c                   pop    %r12
  40249c:       5d                      pop    %rbp
  40249d:       c3                      ret
  40249e:       66 90                   xchg   %ax,%ax
  4024a0:       31 c0                   xor    %eax,%eax
  4024a2:       48 8d 4d e8             lea    -0x18(%rbp),%rcx
  4024a6:       48 8d 55 e4             lea    -0x1c(%rbp),%rdx
  4024aa:       be 86 31 40 00          mov    $0x403186,%esi
  4024af:       48 89 df                mov    %rbx,%rdi
  4024b2:       e8 29 ed ff ff          call   4011e0 <__isoc99_sscanf@plt>
  4024b7:       83 f8 02                cmp    $0x2,%eax
  4024ba:       75 d7                   jne    402493 <phase_05+0x23>
  4024bc:       48 8d 75 ec             lea    -0x14(%rbp),%rsi
  4024c0:       4c 89 e7                mov    %r12,%rdi
  4024c3:       e8 38 ff ff ff          call   402400 <p5_traverse>
  4024c8:       89 c2                   mov    %eax,%edx
  4024ca:       31 c0                   xor    %eax,%eax
  4024cc:       39 55 e4                cmp    %edx,-0x1c(%rbp)
  4024cf:       75 c4                   jne    402495 <phase_05+0x25>
  4024d1:       8b 45 ec                mov    -0x14(%rbp),%eax
  4024d4:       39 45 e8                cmp    %eax,-0x18(%rbp)
  4024d7:       0f 94 c0                sete   %al
  4024da:       eb b9                   jmp    402495 <phase_05+0x25>
```

分析汇编代码：

```
lea    -0x14(%rbp),%rsi       # 准备第 2 个参数：变量地址（用来存额外的输出值）
mov    %r12,%rdi              # 准备第 1 个参数：challenge
call   402400 <p5_traverse>   # 调用 p5_traverse(challenge, &out_val)
mov    %eax,%edx              # 返回值放在 %eax（第 1 个数字）

cmp    %edx,-0x1c(%rbp)       # 检查输入的第 1 个数字，是否等于返回值
jne    ...                    # 不相等就爆炸

mov    -0x14(%rbp),%eax       # 取出刚才 p5_traverse 写入的 out_val
cmp    %eax,-0x18(%rbp)       # 检查输入的第 2 个数字，是否等于 out_val
sete   %al                    # 相等就过关
```

`values[0]` = `p5_traverse(challenge, &val)` 的返回值。

`values[1]` = `p5_traverse` 执行过程中修改的 `val` 的值。

对 `p5_traverse` 运行odjump，得到汇编代码如下：

```
0000000000402400 <p5_traverse>:
  402400:       f3 0f 1e fa             endbr64
  402404:       48 89 f8                mov    %rdi,%rax
  402407:       48 89 f9                mov    %rdi,%rcx
  40240a:       49 89 f0                mov    %rsi,%r8
  40240d:       83 e0 07                and    $0x7,%eax
  402410:       48 c1 e9 03             shr    $0x3,%rcx
  402414:       48 8b 14 c5 60 5b 40    mov    0x405b60(,%rax,8),%rdx
  40241b:       00
  40241c:       48 89 f8                mov    %rdi,%rax
  40241f:       48 c1 ef 20             shr    $0x20,%rdi
  402423:       48 c1 e8 06             shr    $0x6,%rax
  402427:       31 f8                   xor    %edi,%eax
  402429:       8b 72 08                mov    0x8(%rdx),%esi
  40242c:       89 cf                   mov    %ecx,%edi
  40242e:       35 a5 55 00 00          xor    $0x55a5,%eax
  402433:       03 42 0c                add    0xc(%rdx),%eax
  402436:       8d 04 80                lea    (%rax,%rax,4),%eax
  402439:       41 89 30                mov    %esi,(%r8)
  40243c:       31 f0                   xor    %esi,%eax
  40243e:       83 e7 03                and    $0x3,%edi
  402441:       74 1d                   je     402460 <p5_traverse+0x60>
  402443:       b9 01 00 00 00          mov    $0x1,%ecx
  402448:       48 8b 12                mov    (%rdx),%rdx
  40244b:       83 c1 01                add    $0x1,%ecx
  40244e:       8b 72 08                mov    0x8(%rdx),%esi
  402451:       03 42 0c                add    0xc(%rdx),%eax
  402454:       8d 04 80                lea    (%rax,%rax,4),%eax
  402457:       41 89 30                mov    %esi,(%r8)
  40245a:       31 f0                   xor    %esi,%eax
  40245c:       39 cf                   cmp    %ecx,%edi
  40245e:       73 e8                   jae    402448 <p5_traverse+0x48>
  402460:       c3                      ret
```

对应代码如下：

```
struct Node {
    struct Node *next; // 偏移 0x0
    uint32_t val8;     // 偏移 0x8
    uint32_t val12;    // 偏移 0xc
};

int p5_traverse(uint64_t challenge, uint32_t *out_val) {

    uint32_t idx = challenge & 0x7;
    uint32_t rcx_val = challenge >> 3;
    struct Node *node = node_array[idx];

    uint32_t eax = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    eax ^= 0x55a5;
    eax += node->val12;
    eax = eax * 5;
    eax ^= node->val8;

    *out_val = node->val8;
```

```
    uint32_t steps = rcx_val & 0x3;

    for (uint32_t i = 1; i <= steps; i++) {
        node = node->next;
        eax += node->val12;
        eax = eax * 5;
        eax ^= node->val8;
        *out_val = node->val8;
    }

    return eax;
}
```

用odjump查看指针数组，用gdb查看具体数据。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ objdump -s -j .data bomb

bomb:     file format elf64-x86-64

Contents of section .data:
 408088 00000000 00000000 00000000 00000000  ................
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ gdb ./bomb
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./bomb...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
(No debugging symbols found in ./bomb)
(gdb) x/8gx 0x405b60
0x405b60 <p5_heads>:    0x0000000000405c50      0x0000000000405ba0
0x405b70 <p5_heads+16>: 0x0000000000405bd0      0x0000000000405c20
0x405b80 <p5_heads+32>: 0x0000000000405c00      0x0000000000405bc0
0x405b90 <p5_heads+48>: 0x0000000000405c30      0x0000000000405bb0
(gdb) x/12xw 0x405c80
0x405c80 <p5_agent_urgent+32>:  0x27746e65      0x72672073      0x64656461      0x73736120
0x405c90 <p5_agent_urgent+48>:  0x6d6e6769      0x2e746e65      0x6f745320      0x6f6e2070
0x405ca0 <p5_agent_urgent+64>:  0x64203b77      0x6f6e206f      0x72772074      0x20657469
(gdb)
```

获得链表头指针

```
0x405c50
0x405ba0
0x405bd0
0x405c20
0x405c00
0x405bc0
0x405c30
0x405bb0
```

发现 `0x405c80` 过度偏离，为查找 `0x405ba0` 到 `0x405c50` 之间所有节点，输入如下gdb命令：`x/24gx 0x405ba0`，得到如下链表数据：

```
(gdb) x/8gx 0x405b60
0x405b60 <p5_heads>:    0x0000000000405c50      0x0000000000405ba0
0x405b70 <p5_heads+16>: 0x0000000000405bd0      0x0000000000405c20
0x405b80 <p5_heads+32>: 0x0000000000405c00      0x0000000000405bc0
0x405b90 <p5_heads+48>: 0x0000000000405c30      0x0000000000405bb0
(gdb) x/12xw 0x405c80
0x405c80 <p5_agent_urgent+32>:  0x27746e65      0x72672073      0x64656461      0x73736120
0x405c90 <p5_agent_urgent+48>:  0x6d6e6769      0x2e746e65      0x6f745320      0x6f6e2070
0x405ca0 <p5_agent_urgent+64>:  0x64203b77      0x6f6e206f      0x72772074      0x20657469
(gdb) x/24gx 0x405ba0
0x405ba0 <p5_nodes>:    0x0000000000405bd0      0x0000f8e80000022f
0x405bb0 <p5_nodes+16>: 0x0000000000405c40      0x0000c61b00000784
0x405bc0 <p5_nodes+32>: 0x0000000000405c30      0x0000627400000411
0x405bd0 <p5_nodes+48>: 0x0000000000405c20      0x00002ecf0000062a
0x405be0 <p5_nodes+64>: 0x0000000000405bf0      0x0000f92900000982
0x405bf0 <p5_nodes+80>: 0x0000000000000000      0x0000a57b00000b6a
0x405c00 <p5_nodes+96>: 0x0000000000405bc0      0x0000558c00000104
0x405c10 <p5_nodes+112>:        0x0000000000405be0      0x00001ef60000034d
0x405c20 <p5_nodes+128>:        0x0000000000405c00      0x00001bc400000043
0x405c30 <p5_nodes+144>:        0x0000000000405bb0      0x00003b8700000542
0x405c40 <p5_nodes+160>:        0x0000000000405c10      0x000089840000081d
0x405c50 <p5_nodes+176>:        0x0000000000405ba0      0x0000dc6c00000a7c
(gdb)
```

由于小端序（Little-Endian）的存在，低位数据存在低地址。例如，`0x0000222f` 是 `val8`；`0x0000f8e8` 是 `val12`。

逐一映射为数组索引，头指针地址换为下标，得到 `heads` 数组：`{ 11, 0, 3, 8, 6, 2, 9, 1 }`；节点 `next` 指针替换成下标，得到 `nodes` 数组。

综上，`challenge & 0x7` 决定起始节点，`steps = (challenge >> 3) & 0x3` 决定在链表上走的步数。把每一步的 `val12` 加上去，乘以 5，再异或 `val8`。每走到一个新节点，都把当前的 `val8` 记录下来（存进 `values[1]`）。

转换为C代码如下：

```c
int solve_phase_5(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 2;

    typedef struct {
        int next;
        uint32_t val8;
        uint32_t val12;
    } Node;


    static const Node nodes[] = {
        { 3,  0x0000222f, 0x0000f8e8 }, // 0: 0x405ba0
        { 10, 0x00000784, 0x0000c61b }, // 1: 0x405bb0
        { 9,  0x00000411, 0x00006274 }, // 2: 0x405bc0
        { 8,  0x0000062a, 0x00002ecf }, // 3: 0x405bd0
        { 5,  0x00000982, 0x0000f929 }, // 4: 0x405be0
        { -1, 0x00000b6a, 0x0000a57b }, // 5: 0x405bf0
        { 2,  0x00000104, 0x0000558c }, // 6: 0x405c00
        { 4,  0x0000034d, 0x00001ef6 }, // 7: 0x405c10
        { 6,  0x00000443, 0x00001bc0 }, // 8: 0x405c20
        { 1,  0x00000542, 0x00003b87 }, // 9: 0x405c30
        { 7,  0x0000081d, 0x00008984 }, // 10: 0x405c40
        { 0,  0x00000a7c, 0x0000dc6c }  // 11: 0x405c50
    };
```

```
    static const int heads[] = { 11, 0, 3, 8, 6, 2, 9, 1 };

    uint32_t idx = challenge & 0x7;
    uint32_t rcx_val = challenge >> 3;
    uint32_t steps = rcx_val & 0x3;

    int node_idx = heads[idx];

    uint32_t eax = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);
    eax ^= 0x55a5;
    eax += nodes[node_idx].val12;
    eax = eax * 5;
    eax ^= nodes[node_idx].val8;

    values[1] = nodes[node_idx].val8;

    for (uint32_t i = 1; i <= steps; i++) {
        node_idx = nodes[node_idx].next;
        eax += nodes[node_idx].val12;
        eax = eax * 5;
        eax ^= nodes[node_idx].val8;
        values[1] = nodes[node_idx].val8;
    }

    values[0] = eax;

    return write_answer(out, cap, values, count);
}
```

编译执行成功。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
117 579 2911 14539 72677 363391
3 3625181654
2544721993
826641360 2684
Phase 6 solver is incomplete or failed.
```

phase_06

运行odjump，得到汇编代码如下：

```
00000000004025a0 <phase_06>:
  4025a0:       f3 0f 1e fa             endbr64
  4025a4:       55                      push   %rbp
  4025a5:       48 89 e5                mov    %rsp,%rbp
  4025a8:       41 54                   push   %r12
  4025aa:       49 89 f4                mov    %rsi,%r12
  4025ad:       be 01 00 00 00          mov    $0x1,%esi
  4025b2:       53                      push   %rbx
  4025b3:       48 89 fb                mov    %rdi,%rbx
  4025b6:       48 83 ec 10             sub    $0x10,%rsp
  4025ba:       e8 91 06 00 00          call   402c50 <valid_u32_fields>
  4025bf:       84 c0                   test   %al,%al
  4025c1:       75 0d                   jne    4025d0 <phase_06+0x30>
  4025c3:       48 83 c4 10             add    $0x10,%rsp
  4025c7:       31 c0                   xor    %eax,%eax
  4025c9:       5b                      pop    %rbx
  4025ca:       41 5c                   pop    %r12
  4025cc:       5d                      pop    %rbp
  4025cd:       c3                      ret
  4025ce:       66 90                   xchg   %ax,%ax
  4025d0:       31 c0                   xor    %eax,%eax
  4025d2:       48 8d 55 ec             lea    -0x14(%rbp),%rdx
  4025d6:       be 89 31 40 00          mov    $0x403189,%esi
  4025db:       48 89 df                mov    %rbx,%rdi
  4025de:       e8 fd eb ff ff          call   4011e0 <__isoc99_sscanf@plt>
  4025e3:       83 f8 01                cmp    $0x1,%eax
  4025e6:       75 db                   jne    4025c3 <phase_06+0x23>
  4025e8:       4c 89 e7                mov    %r12,%rdi
  4025eb:       e8 60 ff ff ff          call   402550 <p6_dispatch>
  4025f0:       39 45 ec                cmp    %eax,-0x14(%rbp)
  4025f3:       0f 94 c0                sete   %al
  4025f6:       48 83 c4 10             add    $0x10,%rsp
  4025fa:       5b                      pop    %rbx
  4025fb:       41 5c                   pop    %r12
  4025fd:       5d                      pop    %rbp
  4025fe:       c3                      ret

Disassembly of section .fini:
```

分析汇编代码。

```
#检查输入
mov     $0x1,%esi
call    402c50 <valid_u32_fields>
#读取输入
call    4011e0 <__isoc99_sscanf@plt>  # 把输入转成整数，存到 -0x14(%rbp)
#计算比较
mov     %r12,%rdi             # 第 1 个参数：challenge
call    402550 <p6_dispatch>  # 调用 p6_dispatch(challenge)
cmp     %eax,-0x14(%rbp)      # 比较输入的数字和 p6_dispatch 的返回值
sete    %al                   # 相等就过关，否则爆炸
```

对`p6_dispatch`执行odjump，得到如下汇编代码：

```
0000000000402550 <p6_dispatch>:
  402550:       f3 0f 1e fa             endbr64
  402554:       55                      push   %rbp
  402555:       48 89 e5                mov    %rsp,%rbp
  402558:       53                      push   %rbx
  402559:       48 89 fb                mov    %rdi,%rbx
  40255c:       48 c1 ef 06             shr    $0x6,%rdi
  402560:       48 89 d8                mov    %rbx,%rax
  402563:       48 c1 e8 20             shr    $0x20,%rax
  402567:       31 c7                   xor    %eax,%edi
  402569:       48 89 d8                mov    %rbx,%rax
  40256c:       48 83 ec 08             sub    $0x8,%rsp
  402570:       83 e3 38                and    $0x38,%ebx
  402573:       83 e0 07                and    $0x7,%eax
  402576:       ff 14 c5 00 5e 40 00    call   *0x405e00(,%rax,8)
  40257d:       35 f9 b8 00 00          xor    $0xb8f9,%eax
  402582:       89 c7                   mov    %eax,%edi
  402584:       ff 93 00 5e 40 00       call   *0x405e00(%rbx)
  40258a:       48 8b 5d f8             mov    -0x8(%rbp),%rbx
  40258e:       c9                      leave
  40258f:       05 01 14 00 00          add    $0x1401,%eax
  402594:       c3                      ret

Disassembly of section .fini:
```

翻译为伪代码如下：

```
uint32_t arg = (challenge >> 6) ^ (challenge >> 32);
uint32_t idx1 = challenge & 0x7;
uint32_t idx2 = (challenge >> 3) & 0x7;

uint32_t res1 = func_table[idx1](arg);

res1 ^= 0xb8f9;
uint32_t res2 = func_table[idx2](res1);

return res2 + 0x1401;
```

`func_table` 是包含8个函数指针的数组，打印其地址。

```
(gdb) x/8gx 0x405e00
0x405e00 <p6_ops>:      0x0000000000402510      0x00000000004024f0
0x405e10 <p6_ops+16>:   0x00000000004024e0      0x0000000000402540
0x405e20 <p6_ops+32>:   0x0000000000402500      0x0000000000402540
0x405e30 <p6_ops+48>:   0x0000000000402530      0x00000000004024e0
```

两个索引重复，说明函数为6个。

```
00000000004024e0 <p6_op_0>:
  4024e0:       f3 0f 1e fa             endbr64
  4024e4:       89 f8                   mov    %edi,%eax
  4024e6:       35 38 bc 00 00          xor    $0xbc38,%eax
  4024eb:       8d 04 f8                lea    (%rax,%rdi,8),%eax
  4024ee:       c3                      ret
  4024ef:       90                      nop

00000000004024f0 <p6_op_1>:
  4024f0:       f3 0f 1e fa             endbr64
  4024f4:       89 f8                   mov    %edi,%eax
  4024f6:       c1 c8 03                ror    $0x3,%eax
  4024f9:       c3                      ret
  4024fa:       66 0f 1f 44 00 00       nopw   0x0(%rax,%rax,1)

0000000000402500 <p6_op_2>:
  402500:       f3 0f 1e fa             endbr64
  402504:       89 f8                   mov    %edi,%eax
  402506:       c1 c0 03                rol    $0x3,%eax
  402509:       35 87 53 00 00          xor    $0x5387,%eax
  40250e:       c3                      ret
  40250f:       90                      nop

0000000000402510 <p6_op_3>:
  402510:       f3 0f 1e fa             endbr64
  402514:       8d 84 3f 3a e8 00 00    lea    0xe83a(%rdi,%rdi,1),%eax
  40251b:       8d 14 bf                lea    (%rdi,%rdi,4),%edx
  40251e:       31 d0                   xor    %edx,%eax
  402520:       c3                      ret
  402521:       66 66 2e 0f 1f 84 00    data16 cs nopw 0x0(%rax,%rax,1)
  402528:       00 00 00 00
  40252c:       0f 1f 40 00             nopl   0x0(%rax)

0000000000402530 <p6_op_4>:
  402530:       f3 0f 1e fa             endbr64
  402534:       8d 87 4b 53 00 00       lea    0x534b(%rdi),%eax
  40253a:       c1 ef 05                shr    $0x5,%edi
  40253d:       31 f8                   xor    %edi,%eax
  40253f:       c3                      ret
```

```
0000000000402540 <p6_op_5>:
  402540:       f3 0f 1e fa             endbr64
  402544:       89 f8                   mov    %edi,%eax
  402546:       d1 e8                   shr    $1,%eax
  402548:       31 f8                   xor    %edi,%eax
  40254a:       05 65 e3 00 00          add    $0xe365,%eax
  40254f:       c3                      ret
```

算子函数对应的C代码如下：

1. `p6_op_0` **(0x4024e0)**: `(x ^ 0xbc38) + (x * 8)`
2. `p6_op_1` **(0x4024f0)**: `(x >> 3) | (x << 29)`
3. `p6_op_2` **(0x402500)**: `((x << 3) | (x >> 29)) ^ 0x5387`
4. `p6_op_3` **(0x402510)**: `((x * 2) + 0xe83a) ^ (x * 5)`
5. `p6_op_4` **(0x402530)**: `(x + 0x534b) ^ (x >> 5)`
6. `p6_op_5` **(0x402540)**: `((x >> 1) ^ x) + 0xe365`

最终得到C代码如下：

```c
int solve_phase_6(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    uint32_t arg = (uint32_t)(challenge >> 6) ^ (uint32_t)(challenge >> 32);

    uint32_t idx1 = challenge & 0x7;
    uint32_t res1 = 0;
    switch (idx1) {
        case 0: res1 = ((arg * 2) + 0xe83a) ^ (arg * 5); break;
        case 1: res1 = (arg >> 3) | (arg << 29);break;
        case 2: res1 = (arg ^ 0xbc38) + (arg * 8);break;
        case 3: res1 = ((arg >> 1) ^ arg) + 0xe365;break;
        case 4: res1 = ((arg << 3) | (arg >> 29)) ^ 0x5387;break;
        case 5: res1 = ((arg >> 1) ^ arg) + 0xe365;break;
        case 6: res1 = (arg + 0x534b) ^ (arg >> 5);break;
        case 7: res1 = (arg ^ 0xbc38) + (arg * 8);break;
    }

    res1 ^= 0xb8f9;

    uint32_t idx2 = (challenge >> 3) & 0x7;
    uint32_t res2 = 0;
    switch (idx2) {
        case 0: res2 = ((res1 * 2) + 0xe83a) ^ (res1 * 5);break;
        case 1: res2 = (res1 >> 3) | (res1 << 29);break;
        case 2: res2 = (res1 ^ 0xbc38) + (res1 * 8);break;
        case 3: res2 = ((res1 >> 1) ^ res1) + 0xe365;break;
        case 4: res2 = ((res1 << 3) | (res1 >> 29)) ^ 0x5387;break;
        case 5: res2 = ((res1 >> 1) ^ res1) + 0xe365;break;
        case 6: res2 = (res1 + 0x534b) ^ (res1 >> 5);break;
        case 7: res2 = (res1 ^ 0xbc38) + (res1 * 8);break;
    }

    values[0] = res2 + 0x1401;
    return write_answer(out, cap, values, 1);
}
```

编译执行完成。

```
oliviaistsehrgut@localhost:~/coding/ICS26fall/lab/BombLab$ ./defuser
1943466400
117 579 2911 14539 72677 363391
3 3625181654
2544721993
826641360 2684
3510704637
```

拆除炸弹过程中，第三关失败。

```
[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x71c18690ee42c90b
Input (unsigned decimal integers):
3 3625181654
BOOM!!!
[BOOM] 本次维护中断。
```

启用gdb断点，获取第三关正确答案。

```
[1 已解除] [2 已解除] [3 当前] [4 待解除] [5 待解除] [6 待解除]

第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x71c18690ee42c90b
Input (unsigned decimal integers):
3 3625181654

Breakpoint 1, 0x00000000004022fe in phase_03 ()
(gdb) p/x $eax
$1 = 0xa20e8316
(gdb)
```

发现映射关系处理错误，输入3应当进入`p3_case_5`，调整语序，得到新代码。

```c
int solve_phase_3(uint64_t challenge, char *out, size_t cap) {
    uint32_t values[MAX_FIELDS] = {0};
    size_t count = 0;
        values[0] = challenge & 0x7;

    uint32_t x = (uint32_t)(challenge >> 32) ^ (uint32_t)(challenge >> 3);
    uint32_t ans = 0;

    switch (values[0]) {
        case 0:
            ans = (x * 2) ^ 0x5eb4;

            break;
        case 1:
            ans = (x ^ 0x1cdb) + (x >> 1);
            break;
        case 2:
            ans = (x * 4) ^ 0x1ca4;

            break;
        case 3:
```

```
            ans = (x ^ 0x4c8f) + (x >> 1);
            break;
        case 4:
            ans = (x * 5) + 0x424dc;
            break;
        case 5:
            ans = (x * 3) + 0xb0c9;
            break;
        case 6:
            ans = (x * 5) + 0x888a;
            break;
        case 7:
            ans = (x * 5) + 0x3c53c;
            break;
    }

    values[1] = ans;
    count = 2;
    return write_answer(out, cap, values, count);
}
```

成功拆除。

```
第 3 关 · 分流站
沿着钟楼的线路，你们来到街口的分流站。
屏幕上的请求到达一个路口，又被送回前一个路口。
林夏在纸上划掉了一条重复路线：‘它一直在旧地址之间绕。’
站里保留着几种不同的处理办法。请求的标记决定它会走哪一条。

Phase 3 challenge: 0x71c18690ee42c90b
Input (unsigned decimal integers):
Phase 3 defused.

[1 待解除] [2 待解除] [3 已解除] [4 待解除] [5 待解除] [6 待解除]
```

使用defuser全部拆除(•̀ω•́)y

```
[1 已解除] [2 已解除] [3 已解除] [4 已解除] [5 已解除] [6 已解除]

Congratulations! You've defused the bomb.
[SAFE] 六项检查全部通过。新存档已启用。

你们走回街上。送货司机已经发动了车：
‘昨天耽误了。今天得早点走。’
维修铺的灯装好了，老板正把那张写着‘明天装上’的纸条揭下来。
林夏站在橱窗前看了一会儿，跟上了你。

退出前，终端收到 sky 的消息：
‘日志看到了，新存档正常。谢谢你们，剩下的维护我来接。’

实验室的灯还亮着，外套仍搭在椅背上。你确认文件都保存了，合上电脑。
林夏揉了揉眼睛：‘还活着，精神状态另说。’
你把椅子推回桌下：‘事已至此，先睡觉吧。’
你们关掉灯，走出了实验室。

---

谢谢你远道而来，与我相遇。

晚安。

—— sky
```

secret

对三个secret函数分别运行odjump

```
objdump -d bomb --disassemble=secret_gate
objdump -d bomb --disassemble=secret_copy
objdump -d bomb --disassemble=secret_phase
```

得到汇编代码如下：

gate：

```
0000000000402700 <secret_gate>:
  402700:       f3 0f 1e fa             endbr64
  402704:       55                      push   %rbp
  402705:       31 c0                   xor    %eax,%eax
  402707:       b9 14 00 00 00          mov    $0x14,%ecx
  40270c:       48 89 e5                mov    %rsp,%rbp
  40270f:       41 56                   push   %r14
  402711:       49 89 d6                mov    %rdx,%r14
  402714:       41 55                   push   %r13
  402716:       49 89 f5                mov    %rsi,%r13
  402719:       41 54                   push   %r12
  40271b:       4c 8d a5 30 ff ff ff    lea    -0xd0(%rbp),%r12
  402722:       53                      push   %rbx
  402723:       48 89 fb                mov    %rdi,%rbx
  402726:       4c 89 e7                mov    %r12,%rdi
  402729:       48 81 ec b0 00 00 00    sub    $0xb0,%rsp
  402730:       f3 48 ab                rep stos %rax,(%rdi)
  402733:       c7 07 00 00 00 00       movl   $0x0,(%rdi)
  402739:       48 89 df                mov    %rbx,%rdi
  40273c:       e8 4f ea ff ff          call   401190 <strlen@plt>
  402741:       48 83 e8 24             sub    $0x24,%rax
  402745:       48 83 f8 7f             cmp    $0x7f,%rax
  402749:       76 15                   jbe    402760 <secret_gate+0x60>
  40274b:       48 81 c4 b0 00 00 00    add    $0xb0,%rsp
  402752:       31 c0                   xor    %eax,%eax
  402754:       5b                      pop    %rbx
  402755:       41 5c                   pop    %r12
  402757:       41 5d                   pop    %r13
  402759:       41 5e                   pop    %r14
  40275b:       5d                      pop    %rbp
  40275c:       c3                      ret
  40275d:       0f 1f 00                nopl   (%rax)
  402760:       48 89 de                mov    %rbx,%rsi
  402763:       4c 89 e7                mov    %r12,%rdi
  402766:       e8 75 ff ff ff          call   4026e0 <secret_copy>
  40276b:       81 bd 50 ff ff ff 4f    cmpl   $0x4e45504f,-0xb0(%rbp)
  402772:       50 45 4e
  402775:       75 d4                   jne    40274b <secret_gate+0x4b>
  402777:       4c 89 f2                mov    %r14,%rdx
```

```
  40277a:       4c 89 ee                mov    %r13,%rsi
  40277d:       48 8d bd 54 ff ff ff    lea    -0xac(%rbp),%rdi
  402784:       e8 97 fe ff ff          call   402620 <secret_phase>
  402789:       48 81 c4 b0 00 00 00    add    $0xb0,%rsp
  402790:       5b                      pop    %rbx
  402791:       41 5c                   pop    %r12
  402793:       41 5d                   pop    %r13
  402795:       41 5e                   pop    %r14
  402797:       5d                      pop    %rbp
  402798:       c3                      ret
```

copy:

```
00000000004026e0 <secret_copy>:
  4026e0:       f3 0f 1e fa             endbr64
  4026e4:       31 c0                   xor    %eax,%eax
  4026e6:       66 2e 0f 1f 84 00 00    cs nopw 0x0(%rax,%rax,1)
  4026ed:       00 00 00
  4026f0:       0f b6 14 06             movzbl (%rsi,%rax,1),%edx
  4026f4:       88 14 07                mov    %dl,(%rdi,%rax,1)
  4026f7:       48 83 c0 01             add    $0x1,%rax
  4026fb:       84 d2                   test   %dl,%dl
  4026fd:       75 f1                   jne    4026f0 <secret_copy+0x10>
  4026ff:       c3                      ret
```

phase:

```
0000000000402620 <secret_phase>:
  402620:       f3 0f 1e fa             endbr64
  402624:       55                      push   %rbp
  402625:       48 89 e5                mov    %rsp,%rbp
  402628:       41 55                   push   %r13
  40262a:       49 89 d5                mov    %rdx,%r13
  40262d:       41 54                   push   %r12
  40262f:       49 89 f4                mov    %rsi,%r12
  402632:       be 03 00 00 00          mov    $0x3,%esi
  402637:       53                      push   %rbx
  402638:       48 89 fb                mov    %rdi,%rbx
  40263b:       48 83 ec 18             sub    $0x18,%rsp
  40263f:       e8 0c 06 00 00          call   402c50 <valid_u32_fields>
  402644:       84 c0                   test   %al,%al
  402646:       75 18                   jne    402660 <secret_phase+0x40>
  402648:       31 d2                   xor    %edx,%edx
  40264a:       48 83 c4 18             add    $0x18,%rsp
  40264e:       89 d0                   mov    %edx,%eax
  402650:       5b                      pop    %rbx
  402651:       41 5c                   pop    %r12
  402653:       41 5d                   pop    %r13
  402655:       5d                      pop    %rbp
  402656:       c3                      ret
  402657:       66 0f 1f 84 00 00 00    nopw   0x0(%rax,%rax,1)
  40265e:       00 00
  402660:       31 c0                   xor    %eax,%eax
  402662:       4c 8d 45 dc             lea    -0x24(%rbp),%r8
  402666:       48 8d 4d d8             lea    -0x28(%rbp),%rcx
  40266a:       be 83 31 40 00          mov    $0x403183,%esi
  40266f:       48 8d 55 d4             lea    -0x2c(%rbp),%rdx
  402673:       48 89 df                mov    %rbx,%rdi
  402676:       e8 65 eb ff ff          call   4011e0 <__isoc99_sscanf@plt>
  40267b:       83 f8 03                cmp    $0x3,%eax
  40267e:       75 c8                   jne    402648 <secret_phase+0x28>
  402680:       44 89 e9                mov    %r13d,%ecx
  402683:       31 d2                   xor    %edx,%edx
  402685:       83 e1 03                and    $0x3,%ecx
```

```
402688:       39 4d d4                cmp    %ecx,-0x2c(%rbp)
40268b:       75 bd                   jne    40264a <secret_phase+0x2a>
40268d:       4c 89 e6                mov    %r12,%rsi
402690:       44 69 c9 63 72 00 00    imul   $0x7263,%ecx,%r9d
402697:       4d 89 e8                mov    %r13,%r8
40269a:       48 c1 ee 20             shr    $0x20,%rsi
40269e:       49 c1 e8 20             shr    $0x20,%r8
4026a2:       44 31 e6                xor    %r12d,%esi
4026a5:       45 31 e8                xor    %r13d,%r8d
4026a8:       49 c1 ed 10             shr    $0x10,%r13
4026ac:       89 f7                   mov    %esi,%edi
4026ae:       45 31 e9                xor    %r13d,%r9d
4026b1:       44 31 c7                xor    %r8d,%edi
4026b4:       e8 47 ff ff ff          call   402600 <secret_mix>
4026b9:       44 31 c8                xor    %r9d,%eax
4026bc:       39 45 d8                cmp    %eax,-0x28(%rbp)
4026bf:       75 89                   jne    40264a <secret_phase+0x2a>
4026c1:       d1 ee                   shr    $1,%esi
4026c3:       41 8d 94 08 d8 bb 00    lea    0xbbd8(%r8,%rcx,1),%edx
4026ca:       00
4026cb:       01 f2                   add    %esi,%edx
4026cd:       31 d0                   xor    %edx,%eax
4026cf:       3b 45 dc                cmp    -0x24(%rbp),%eax
4026d2:       0f 94 c2                sete   %dl
4026d5:       e9 70 ff ff ff          jmp    40264a <secret_phase+0x2a>
```

分析汇编代码：`secret_gate` 要求输入以 `"OPEN"` 开头，`secret_phase` 要求剩下的部分必须包含 3 个整数，后者必须通过特定公式，用 `challenge` 和 `context` 计算。

注意到调用了 `secret_mix` ，对其执行odjump，得到汇编代码如下

```
0000000000402600 <secret_mix>:
  402600:       f3 0f 1e fa             endbr64
  402604:       89 f8                   mov    %edi,%eax
  402606:       d1 c0                   rol    $1,%eax
  402608:       05 63 72 00 00          add    $0x7263,%eax
  40260d:       35 d8 bb 00 00          xor    $0xbbd8,%eax
  402612:       c3                      ret
```

将输入左移、加上一个常数、异或一个常数。

得到最终C代码：

```c
int solve_secret(uint64_t challenge, uint64_t context, char *out, size_t cap) {
    uint64_t C = challenge;
    uint64_t T = context;

    uint32_t v0 = T & 0x3;

    uint32_t C_lo = (uint32_t)C;
    uint32_t C_hi = (uint32_t)(C >> 32);
    uint32_t e_si = C_hi ^ C_lo;
    uint32_t T_lo = (uint32_t)T;
    uint32_t T_hi = (uint32_t)(T >> 32);
    uint32_t e_r8 = T_hi ^ T_lo;

    uint32_t r9 = (v0 * 0x7263) ^ (uint32_t)(T >> 16);

    uint32_t mix_arg = e_si ^ e_r8;
    uint32_t mix_res = (mix_arg << 1) | (mix_arg >> 31); // rol 1
    mix_res += 0x7263;
    mix_res ^= 0xbbd8;
```

```
    uint32_t v1 = mix_res ^ r9;

    uint32_t e_si_halved = e_si >> 1;
    uint32_t edx = e_r8 + v0 + 0xbbd8 + e_si_halved;
    uint32_t v2 = v1 ^ edx;

    snprintf(out, cap, "OPEN %u %u %u", v0, v1, v2);

    return 0;
}
```